WEB-250 | Understanding PDO Prepared Statements

Topic: Prepared statements with PHP, PDO, and the `wnc_birds` database
Use with: asgn08 — Active Record Directory
Format: Standalone reference lesson and ungraded practice
Prerequisite: A working asgn07 database connection and the `birds` table

1. Introduction — Why prepared statements?

Suppose someone visits a bird details page:

```
bird.php?id=5
```

The page needs to ask MySQL for the bird whose `id` is `5`. The page receives the ID from the URL, so **the value comes from outside your program**.

It might seem natural to write:

```php
// DON'T USE THIS PATTERN: a value from the URL is inserted directly into
SQL.
$id = $_GET['id'];
$sql = "SELECT * FROM birds WHERE id = $id";
```

The problem is that the program is building SQL by mixing **the SQL instructions** with **an external value**. A malicious or unexpected value could change what the database is asked to do. This is one way SQL injection vulnerabilities arise.

A **prepared statement** keeps those two things separate:

1. Write the SQL with a **placeholder** where the changing value belongs.
2. Give PDO the value separately, telling it how to treat that value.
3. Execute the statement.

For example:

```sql
SELECT id, common_name, scientific_name
FROM birds
WHERE id = :id
```

Here, `:id` is a *named placeholder*. It is not a PHP variable and not a database column; it marks a spot for a value that PDO will supply when the statement runs.

The big idea: SQL describes *what to do*. The bound parameter supplies *data to use*. The data is not inserted into the SQL as executable SQL code.

When should I use `prepare()` rather than `query()`?

Situation	Appropriate PDO approach
List all birds with fixed SQL: `SELECT ... FROM birds ORDER BY common_name`	`query()` is fine; there is no external value.
Find one bird using `?id=...` from the URL	Use `prepare()` with a bound ID.
Search by a name, habitat, or season typed into a form	Use `prepare()` with bound search values.

Prepared statements are a strong default whenever a query uses changing values. They **do not replace** validation of user input or HTML escaping when displaying results.

---

2. Guided example — Find one bird using a bound ID

This example uses the same `birds` table as asgn07 and asgn08. Its columns include `id`, `common_name`, `scientific_name`, `family_name`, `primary_habitat`, `seasonal_status`, and `description`.

The following fragment assumes `$pdo` is an existing connection to `wnc_birds` and `$id` is a validated, positive integer.

```
// Step 1: Put a named placeholder in the SQL.
$sql = 'SELECT id, common_name, scientific_name
        FROM birds
        WHERE id = :id
        LIMIT 1';

// Step 2: Ask PDO to prepare the SQL.
$statement = $pdo->prepare($sql);

// Step 3: Give the placeholder its value and type.
$statement->bindValue(':id', $id, PDO::PARAM_INT);

// Step 4: Run the prepared statement.
$statement->execute();

// Step 5: Fetch one row as an associative array.
$row = $statement->fetch(PDO::FETCH_ASSOC);

// If the record doesn't exist, fetch() returns false.
if ($row === false) {
    print 'That bird was not found.';
} else {
    print htmlspecialchars(
        (string) $row['common_name'],
        ENT_QUOTES | ENT_SUBSTITUTE,
        'UTF-8'
```

```
    );
}
```

What each PDO method does

PHP expression	Meaning
`$pdo->prepare($sql)`	Creates a prepared statement from the SQL containing `:id`.
`$statement->bindValue(':id', $id, PDO::PARAM_INT)`	Associates `:id` with the current value of `$id`, explicitly treating it as an integer.
`$statement->execute()`	Executes the prepared query with its bound value.
`$statement->fetch(PDO::FETCH_ASSOC)`	Reads one result as an associative array, or returns `false` if no row exists.

Notice that `bindValue()` does **not** use a dollar sign in `':id'`: the placeholder belongs to the SQL statement. `$id`, with its dollar sign, is the PHP variable holding the actual number.

Important distinction: `PDO::PARAM_INT` specifies a database parameter type; it is not a substitute for checking that a visitor supplied a valid, positive bird ID.

---

3. Working sandbox — Try it in a browser (ungraded)

Create a new file in your asgn08 copy:

```
web250/asgn08/public/sandbox_prepared_statements.php
```

Keep your existing asgn07 `private/initialize.php` and `private/classes/Database.php`. **Do not replace your connection configuration** for this lesson. The following example works whether `initialize.php` supplies `$pdo` or your existing `Database::connect()` method supplies it.

Copy this **entire file** into `sandbox_prepared_statements.php`:

```php
<?php

declare(strict_types=1);

require_once dirname(__DIR__) . '/private/initialize.php';

function prepared_h(string $value): string
{
    return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
```

```php
$bird = null;
$message = 'Enter a bird ID from your database.';

// Run the lookup only after the visitor submits the form.
if (array_key_exists('id', $_GET)) {

    // Reject text, zero, and negative numbers before querying the
database.
    $id = filter_input(
        INPUT_GET,
        'id',
        FILTER_VALIDATE_INT,
        ['options' => ['min_range' => 1]]
    );

    if (!is_int($id)) {
        $message = 'Please enter a positive whole-number bird ID.';
    } else {
        try {
            // Reuse the existing asgn07 PDO connection.
            $pdo = (isset($pdo) && $pdo instanceof PDO)
                ? $pdo
                : Database::connect();

            // The SQL is a fixed template. The ID is NOT concatenated into
it.
            $sql = 'SELECT id, common_name, scientific_name,
                           primary_habitat
                    FROM birds
                    WHERE id = :id
                    LIMIT 1';

            $statement = $pdo->prepare($sql);
            $statement->bindValue(':id', $id, PDO::PARAM_INT);
            $statement->execute();

            // A row is an associative array, not yet a Bird object.
            $row = $statement->fetch(PDO::FETCH_ASSOC);

            if ($row === false) {
                $message = 'No bird was found with that ID.';
            } else {
                $bird = $row;
                $message = '';
            }
        } catch (PDOException $e) {
            // Log details for developers, but not for website visitors.
            error_log($e->getMessage());
            $message = 'The bird database is temporarily unavailable.';
        }
    }
}
?>
<!doctype html>
```

```
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Bird Prepared Statement Sandbox</title>
</head>
<body>
    <h1>Find a Bird by ID</h1>

    <form method="get" action="">
        <label for="id">Bird ID:</label>
        <input type="number" name="id" id="id" min="1" required>
        <button type="submit">Find bird</button>
    </form>

    <?php if ($message !== ''): ?>
        <p><?= prepared_h($message) ?></p>
    <?php endif; ?>

    <?php if ($bird !== null): ?>
        <h2><?= prepared_h((string) $bird['common_name']) ?></h2>
        <p><strong>ID:</strong> <?= (int) $bird['id'] ?></p>
        <p><strong>Scientific name:</strong>
            <?= prepared_h((string) $bird['scientific_name']) ?>
        </p>
        <p><strong>Habitat:</strong>
            <?= prepared_h((string) $bird['primary_habitat']) ?>
        </p>
    <?php endif; ?>

    <p><a href="index.php">Back to the asgn08 index</a></p>
</body>
</html>
```

Run it

1. Start Apache and MySQL using MAMP (Mac) or XAMPP (Windows).
2. Open the new page in a browser. If your course Apache port is `55000`, the URL is:

   ```
   http://localhost:55000/web250/asgn08/public/sandbox_prepared_statements.php
   ```

   If your working port differs, use that port instead.
3. Find a valid ID in the `birds` table or on your existing bird directory. **Do not assume a particular bird has ID 1.**
4. Enter that ID and click **Find bird**. The bird's common name, scientific name, and habitat should appear.

5. Enter an ID that does not exist. The page should display **No bird was found with that ID.**
6. To test invalid input without the number field blocking it, edit the URL to end with `?id=abc`, `?id=0`, or `?id=-2`. You should see the validation message, not a database error.

Trace the request

```
Browser: ?id=5
    |
    v
Validate that 5 is a positive integer
    |
    v
PDO::prepare('SELECT ... WHERE id = :id')
    |
    v
bindValue(':id', 5, PDO::PARAM_INT)
    |
    v
execute() --> MySQL
    |
    v
fetch(PDO::FETCH_ASSOC) --> row or false
    |
    v
Escape text and display it in HTML
```

---

4. Experiments — Predict, change, run, explain

These exercises are **not graded**. Make a prediction before each experiment; restore the working code afterward.

Experiment A — Remove a row from the result (temporarily). In the SQL `SELECT` list, remove `primary_habitat`, but leave the HTML line that tries to display it. Predict the warning or behavior. Restore the column after testing. **Question:** How does the associative array get its keys?

Experiment B — Change the placeholder name. Change `WHERE id = :id` to `WHERE id = :bird_id`, but **do not** change the `bindValue()` call on the first attempt. Predict what will happen when you run the query. Then fix the placeholder mismatch and confirm the page works again. **Question:** Why must placeholder names match?

Experiment C — Use another changing value. Look up the actual `primary_habitat` value for one of your records and try a separate prepared query:

```php
$habitat = 'REPLACE WITH A HABITAT FROM YOUR DATA';
$sql = 'SELECT common_name FROM birds WHERE primary_habitat = :habitat';
$statement = $pdo->prepare($sql);
$statement->bindValue(':habitat', $habitat, PDO::PARAM_STR);
```

```
$statement->execute();
$rows = $statement->fetchAll(PDO::FETCH_ASSOC);
```

Predict whether this returns **one row, several rows, or no rows**. Note that `PDO::PARAM_STR` is appropriate for text, while `PDO::PARAM_INT` was appropriate for our integer ID.

Experiment D — Compare `fetch()` and `fetchAll()`. In a copy of Experiment C, replace `fetchAll()` with `fetch()`. **Question:** How does this change the type and number of records your code receives? What happens if no records match?

---

5. Checkpoints — Can you explain it?

1. **What is `:id`?** A named placeholder in SQL; PDO binds a value to it.
2. **Why use `prepare()` for a URL-supplied ID?** It separates the supplied data from SQL instructions and protects against SQL injection through that value.
3. **Why validate the ID if you already use a prepared statement?** Parameter binding addresses safe query execution; validation checks whether the input is appropriate (a positive integer ID).
4. **What does `execute()` do?** It runs the prepared SQL using the parameters already bound to the statement.
5. **What does `fetch()` return when there is no match?** `false`, not an empty `Bird` object.
6. **Does a prepared statement make output safe for an HTML page?** No. Escape database text with `htmlspecialchars()` when displaying it.
7. **Can placeholders replace a table or column name?** No. Placeholders represent **data values**, not SQL identifiers or SQL syntax. Choose table/column names from trusted, hard-coded SQL or a carefully controlled allowlist.

---

6. How this connects to asgn08

Your asgn08 `Bird::find_by_id(int $id): ?Bird` method uses exactly this technique: prepare a query containing `WHERE id = :id`, bind the integer ID, run the query, and fetch **one row**.

The additional asgn08 step is **hydration**:

```
One database row (associative array)
               |
               v
     Bird::from_row($row)
               |
               v
        One Bird object
```

If `fetch()` returns `false`, `find_by_id()` returns `null`. If a row exists, pass it through your private `from_row()` helper to create a `Bird` object. This keeps the SQL and object creation in the **model**, while `public/bird.php` handles input, HTTP responses, and HTML output.

Keep the responsibilities distinct:

- **Validation:** Is the input a positive integer ID?
- **Prepared SQL:** Does the database receive the ID as a value rather than SQL code?
- **Hydration:** Is the returned row converted into a `Bird` object?
- **Escaping:** Is the object's text safe to display in HTML?

These four steps solve **different problems**. A reliable PHP application uses all four where appropriate.

Further reference

- PHP manual — PDO::prepare
- PHP manual — PDOStatement::bindValue
- PHP manual — PDOStatement::execute
- PHP manual — Prepared statements and stored procedures

> **Scope:** This file is an explanatory companion for asgn08, not a replacement for its instructions or an additional graded assignment.